

**UNIVERSIDAD NACIONAL DE INGENIERÍA**  
**FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA**



ALGORITMOS Y ESTRUCTURA DE DATOS I - BMA09

INFORME

**“ALGORITMOS GREEDY”**

**Integrantes Del Grupo:**

- Chavez Chavez Fabricio Mathias 20261271B
- Pozo Tacas David Yeshua 20252541K

**Docentes:**

- M.Sc. Luis Antonio Selis Vasquez

**Sección: T**

## **1. Introducción:**

En computación, los algoritmos son herramientas esenciales de programación utilizadas para resolver problemas de manera estructurada, ordenada y eficiente. Existen varias estrategias para crear algoritmos, cada uno diseñado para resolver tipos específicos de problemas basados en necesidades individuales. Estas estrategias incluyen algoritmos Greedy, también llamados algoritmos voraces, que construyen una solución paso a paso eligiendo la mejor opción en cada etapa.

Los algoritmos Greedy son cruciales en la informática debido a su uso en optimización, selección, planificación, problemas de gráficos y asignación de recursos. Funciona tomando una serie de decisiones basadas en una regla específica para construir gradualmente una solución óptima a partir de esas elecciones.

Estudiar esta estrategia nos ayuda a entender cómo se resuelven los problemas algorítmicos y cómo las reglas de selección específicas pueden mejorar o dificultar la solución final. Además, dado que los algoritmos Greedy pueden ser utilizados para resolver muchos problemas diferentes, son una herramienta importante en el análisis de algoritmos.

Este informe cubrirá el estudio de algoritmos codiciosos explicándolo e implementándolo en C. Las definiciones clave para esta estrategia, junto con los algoritmos elegidos para análisis y uso futuro, se definirán a lo largo del informe.

## 2. Fundamento teórico:

### 2.1 ¿Qué es un algoritmo Greedy?

Son algoritmos que construyen una solución de manera progresiva mediante una secuencia de decisiones locales.

En cada etapa se analizan las opciones disponibles y se selecciona aquella que presenta el mayor beneficio o el menor costo (el más óptimo), dependiendo del objetivo del problema. Una vez realizada la elección, el algoritmo continúa trabajando con las alternativas restantes sin modificar normalmente las decisiones tomadas anteriormente.

La idea principal puede expresarse de la siguiente manera:

**“Elegir en cada paso la alternativa que parece ser la mejor en ese momento.”**

Por ejemplo, si un problema busca minimizar costos, una estrategia Greedy podría seleccionar primero la alternativa de menor costo disponible y así sucesivamente.

Sin embargo, es importante comprender que elegir la mejor alternativa disponible en un momento determinado **no garantiza automáticamente que se obtenga la mejor solución global**. Para que Greedy pueda producir una solución óptima, el problema debe cumplir ciertas propiedades específicas que permitan demostrar que las decisiones locales realizadas son correctas.

Por esta razón, un algoritmo Greedy debe diseñarse de acuerdo con las características particulares del problema que se desea resolver ya que a

veces esta solución no es la óptima y se necesitaría de un algoritmo diferente.

## 2.2 Principio de elección Greedy

Como mencionamos anteriormente este principio de elección se basa en construir una elección de manera progresiva tomando en cada etapa **una decisión local**, es decir elegir la alternativa que resulta más conveniente en ese momento de acuerdo a un criterio previamente establecido.

En forma resumida, cuando el algoritmo tiene varias opciones, no analiza las combinaciones que se pueden dar en el futuro simplemente escoge la alternativa más favorable justo en ese momento, esto podría ser una desventaja. Debido a esto también recibe el nombre de voraz(codicioso) ya que intenta conseguir inmediatamente el más beneficioso o el que tiene menor costo basado en el objetivo que desea el problema.

De manera general con ayuda de la matemática vamos a representar a C como el número de opciones disponibles y  $f(x)$  sería el criterio para elegir una de esas opciones y se puede expresar como:

$$x^* = \arg \max_{x \in C} f(x)$$

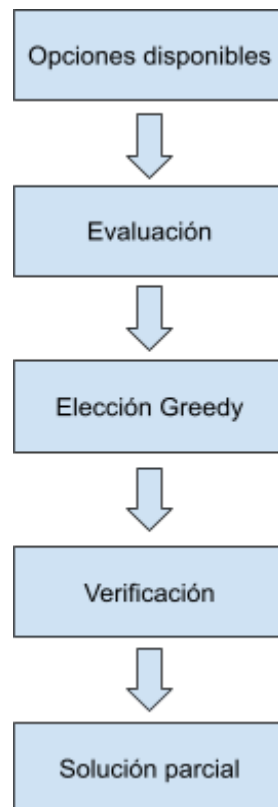
**“De todas las posibles opciones de x que pertenecen a C escogemos el que tenga el mayor valor de x”**

$$x^* = \arg \max_{x \in C} f(x)$$

**“De todas las posibles opciones de  $x$  que pertenecen a  $C$  escogemos el que tenga el menor valor de  $x$ ”**

Una vez elegimos el candidato este pasa a ser parte de una solución al problema si cumple las debidas restricciones de este. Posteriormente el algoritmo actualiza el conjunto de opciones(cambia) y repite el mismo procedimiento en la siguiente etapa así sucesivamente hasta obtener la solución final mediante una secuencia de decisiones locales.

Este proceso podemos representarlo con el siguiente esquema:



Este proceso se repite hasta que la solución quede completa o no quedan más opciones disponibles.

### 2.3 Corrección de la estrategia Greedy

La corrección de una estrategia voraz consiste en demostrar que el procedimiento utilizado por un algoritmo Greedy produce una solución válida. Esta demostración es necesaria porque el hecho de tomar decisiones locales no garantiza por sí solo que el resultado final sea el mejor posible o el más óptimo.

Un algoritmo Greedy construye la solución de manera progresiva paso a paso. Por ello, para demostrar su corrección no basta con analizar únicamente una decisión individual, sino que es necesario demostrar que las decisiones individuales realizadas sucesivamente mantienen la posibilidad de alcanzar una solución óptima hasta completar el problema.

La corrección de una estrategia voraz se fundamenta principalmente en dos ideas: la propiedad de elección Greedy y la subestructura óptima. La primera permite justificar que la elección realizada por el algoritmo puede formar parte de una solución óptima. La segunda permite establecer que, una vez realizada dicha elección, el problema restante puede resolverse de manera óptima.

## 2.4 Propiedad de elección Greedy

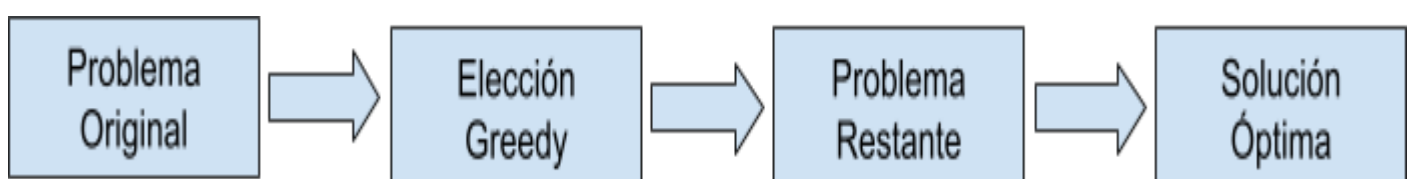
Es una propiedad teórica que permite determinar si una decisión tomada mediante el enfoque Greedy puede conducir a una solución óptima o no. Su idea fundamental es que existe una solución óptima que contiene la elección realizada por el algoritmo.

Esto significa que una elección local no se considera correcta simplemente porque parezca conveniente. Para que pueda ser utilizada de manera justificada, debe ser posible demostrar que tomar esa elección no impide alcanzar el mejor resultado posible para el problema.

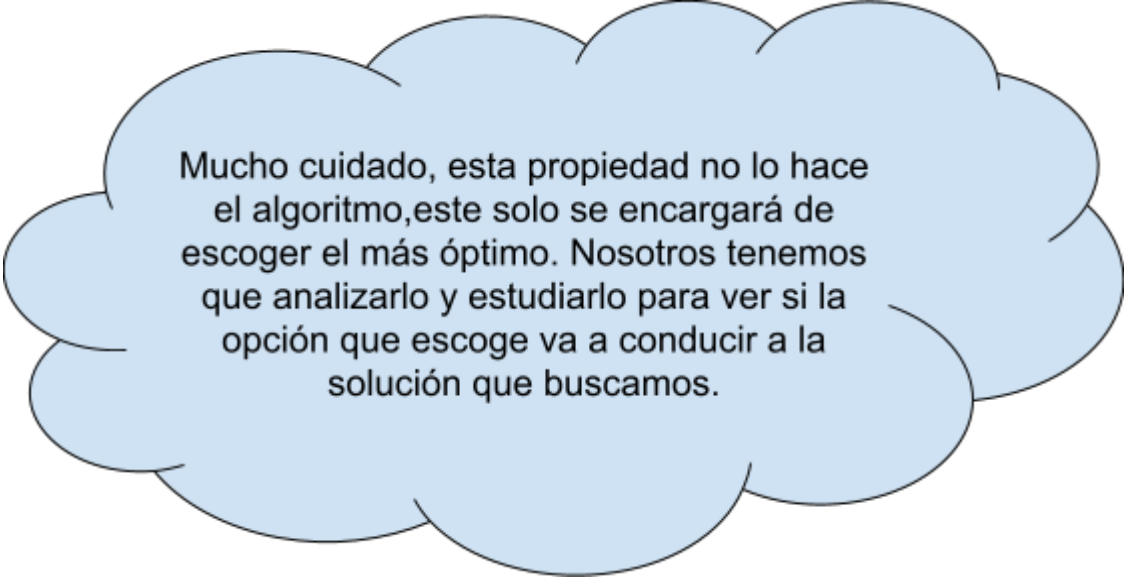
La importancia de esta propiedad se puede entender a partir de la siguiente diferencia. Un algoritmo Greedy puede seleccionar en cada etapa una alternativa que parece ser la mejor según un determinado criterio. Sin embargo, una alternativa que parece conveniente inicialmente podría provocar que posteriormente no sea posible obtener la solución globalmente óptima. Por ello, es necesario establecer si la elección realizada es segura.

Una elección se considera segura cuando puede demostrarse que existe una solución óptima que incluye dicha elección. En consecuencia, el algoritmo puede realizar esa decisión tranquilamente y continuar trabajando con el problema restante sin perder la posibilidad de obtener una solución óptima.

Conceptualmente, la propiedad puede representarse de la siguiente manera:



La idea es que la primera elección realizada por el algoritmo puede encontrarse dentro de alguna solución óptima del problema original. Una vez aceptada esa elección, el problema puede simplificarse y continuar resolviéndose.



Mucho cuidado, esta propiedad no lo hace el algoritmo, este solo se encargará de escoger el más óptimo. Nosotros tenemos que analizarlo y estudiarlo para ver si la opción que escoge va a conducir a la solución que buscamos.

Tomando ideas matemáticas podemos resumirlo en tal expresión:

$$\exists S^* \in OPT \text{ tal que } g \in S^*$$

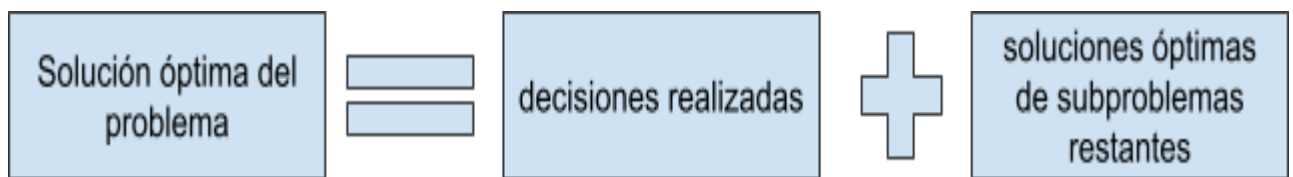
**“Existe al menos una solución óptima( $S^*$ ) que pertenece al conjunto de soluciones OPT y que contiene a la elección Greedy(g)”**

## 2.5 Subestructura óptima

Es una propiedad fundamental en el diseño y análisis de algoritmos de optimización. Un problema presenta subestructura óptima cuando una solución óptima del problema completo puede construirse a partir de soluciones óptimas de problemas más pequeños o subproblemas que forman parte del problema original.

Esta propiedad significa que, al dividir un problema en partes más pequeñas, las soluciones óptimas obtenidas para esas partes pueden utilizarse para construir una solución óptima para el problema general. Por lo tanto, la solución del problema original conserva la optimización de los subproblemas que la componen generando una solución óptima para todo el problema.

De manera general, puede expresarse como:



Esto permite disminuir la complejidad del problema ya que al elegir una solución válida el problema se transforma en una instancia más pequeña del mismo problema.

Este proceso es muy importante en algoritmos Greedy ya que este al tener una elección local considerada segura permite posteriormente concentrarse en el problema restante sin perder la posibilidad de tener una solución óptima del problema. Aunque está por sí sola no es correcta en algoritmos Greedy ya que necesitamos de propiedad de elección Greedy para garantizar que sea correcta.

## 2.6 Ventajas

- Simplicidad: Estos algoritmos suelen tener una implementación sencilla, ya que la construyen paso a paso. Esto facilita su comprensión, programación y mantenimiento.
- Eficiente complejidad computacional: Generalmente los algoritmos Greedy tienden a ser eficientes y muy rápidos en muchos problemas(ojo no en

todos). La complejidad suele ser  $O(n)$  o  $O(n\log(n))$ , especialmente cuando se necesita ordenar los datos primero.

-Bajo consumo de memoria: No necesitan almacenar las soluciones que van obteniendo paso a paso por ende no necesitan una alta cantidad de memoria.

-Puede obtener soluciones óptimas: Cuando el problema posee propiedad de elección Greedy y subestructura óptima podemos garantizar una solución óptima mediante la estrategia Greedy.

## 2.7 Limitaciones

-No siempre encuentras una solución óptima: Una decisión que parece ser la mejor localmente no siempre conducirá a una solución óptima del problema por ende el algoritmo Greedy no se usará en todos los problemas de optimización.

-Necesita de una demostración de corrección: No basta con elegir la alternativa que parezca más conveniente. Es necesario demostrar que dicha elección es segura y que puede formar parte de una solución óptima. Esta demostración suele apoyarse en la propiedad de elección Greedy y la subestructura óptima.

-Puede producir soluciones subóptimas: Cuando el problema no cumple las condiciones necesarias para Greedy, el algoritmo puede encontrar una solución válida, pero que no sea la mejor posible(no es la más óptima). Una decisión tomada al inicio puede limitar las opciones disponibles en etapas posteriores y dificultando la estrategia Greedy.

### 3. Problemas clásicos:

#### 3.1. Selección de Actividades:

Es un problema de planificación de tareas donde se tienen  $n$  actividades que deben usar un recurso que solo puede ser usado por una actividad en cada momento, para la solución de este problema se busca seleccionar la mayor cantidad de actividades que se puedan realizar sin solaparse entre sí. Dos actividades no se solapan si una termina antes de que la otra empiece

##### 3.1.1. Estrategia voraz:

Para la estrategia voraz necesitaremos escoger nuestra elección local.

Los criterios que podríamos escoger para nuestra elección local pueden ser los siguientes:

- Orden creciente de hora de inicio
- Orden creciente de hora de finalización
- Orden creciente de duración

Una vez elegida nuestra elección local, procedemos a seleccionar las actividades según ese criterio, verificando que no se superpongan con las ya seleccionadas. Si la actividad es compatible, se agrega a la solución. Repetimos este proceso hasta recorrer todas las actividades.

##### 3.1.2. Elección local:

Nuestro criterio será elegir la actividad que termine antes y que sea compatible con las ya seleccionadas.

Ejemplo:

se tienen las siguientes actividades:

(1,4) (3,5) (0,6) (5,7) (8,9) (5,9)

Al ordenarlas por tiempo de finalización y eligiendo las que sean compatibles:

(1,4) (5,7) (8,9)

nos queda que podemos realizar 3 actividades como máximo

#### 3.1.3. Propiedad de elección voraz:

Existe una solución óptima que contiene la actividad que termina primero.

Suponiendo que la solución óptima no contiene a la que termina primero, al intercambiarla por ella, el cambio no genera conflictos y no reduce la cantidad de actividades, debido a que el cambio solo reemplaza a la actividad inicial. Entonces se obtiene otra solución óptima que sí contiene a la actividad que termina primero.

#### 3.1.4. Subestructura óptima

Un problema tiene subestructura óptima si una solución óptima del problema contiene soluciones óptimas de subproblemas.

Al elegir la solución óptima, lo que queda por elegir son las actividades restantes que se pueden realizar sin solaparse entre sí y que le siguen a la actividad ya elegida. Es decir, se crea un subproblema del problema inicial para volver a elegir otra solución óptima y determinar las actividades adicionales que se pueden realizar

#### 3.1.5. Corrección de la estrategia voraz:

Se cumplen la propiedad de elección voraz y la subestructura óptima.

Sea “m” la actividad que termina primero.

Sea “O” una solución óptima.

Como “m” termina antes o igual que “mj” podemos reemplazar “m” por “mj” en O creando una nueva solución O’ que tiene la misma cantidad de actividades que O, así que sigue siendo solución óptima.

Por lo tanto, existe una solución óptima que incluye a “m”.

Repitiendo el mismo argumento con el subproblema restante, se concluye que el algoritmo greedy es correcto y produce una solución óptima

#### 3.1.6. Contraejemplo

Otros criterios donde falla el algoritmo greedy serán:

1 Elegir la actividad que empieza primero:

Ejemplo

actividades: (0,10), (1,2),(3,4)

al elegir el que empieza primero solo realizamos una actividad  
(0,10)

Con nuestro criterio greedy elegido óptimo realizaríamos

2 actividades (1,2), (3,4)

2 Elegir la actividad más corta:

Ejemplo

actividades: (0,5), (5,10),(4,6)

Si elegimos la más corta (4,6) solo realizamos una actividad  
con el criterio greedy elegido realizaremos 2 actividades (0,5) y  
(5,10)

#### 3.1.7. Implementación en C

Ingresamos la cantidad de actividades a realizar para poder hallar la máxima cantidad de actividades posibles con el método greedy explicado anteriormente

```
=====
SELECCION DE ACTIVIDADES
=====
Ingrese la cantidad de actividades:
```

Para nuestro ejemplo utilizaremos 8 actividades

```
=====
SELECCION DE ACTIVIDADES
=====
Ingrese la cantidad de actividades: 8

Ingrese el inicio y fin de cada actividad:
(Los tiempos deben ser mayores o iguales a 0 y el inicio <= fin)
```

```
Actividad 1
  Inicio: 1
  Fin:    4
```

```
Actividad 2
  Inicio: 3
  Fin:    5
```

```
Actividad 3
  Inicio: 0
  Fin:    6
```

```
Actividad 4
  Inicio: 5
  Fin:    7
```

```
Actividad 5
  Inicio: 8
  Fin:    9
```

```
Actividad 6
  Inicio: 5
  Fin:    9
```

```
Actividad 7
  Inicio: 6
  Fin:   10
```

```
Actividad 8
  Inicio: 8
  Fin:   11
```

Nuestro resultado por medio de la elección greedy nos indica que la máxima cantidad de actividades que podemos realizar con las actividades presentadas son 3

| ACTIVIDADES SELECCIONADAS |     |
|---------------------------|-----|
| Inicio                    | Fin |
| 1                         | 4   |
| 5                         | 7   |
| 8                         | 9   |
| Total de actividades: 3   |     |

El algoritmo greedy nos garantiza encontrar una solución óptima para hallar la máxima cantidad de actividades posibles a realizar.

#### 3.1.8. Complejidad Computacional

```
int compararActividades(const void *a, const void *b) {
    Actividad *actA = (Actividad *)a;
    Actividad *actB = (Actividad *)b;
    if (actA->fin < actB->fin) return -1;
    if (actA->fin > actB->fin) return 1;
    return 0;
}
```

Tiene complejidad  $O(1)$  porque cada vez que se ejecuta es una llamada individual

```
void seleccionActividades(Actividad actividades[], int n) {
    qsort(actividades, n, sizeof(Actividad), compararActividades);
}
```

Funciona como un quicksort

complejidad computacional de  $O(n \log n)$

```

for (i = 1; i < n; i++) {
    if (actividades[i].inicio >= ultimoFin) {
        printf("    %4d | %4d\n", actividades[i].inicio, actividades[i].fin);
        ultimoFin = actividades[i].fin;
        contador++;
    }
}

```

El bucle se realiza (n-1) veces donde solamente se realiza una comparación con complejidad  $O(1)$

Su complejidad sería de  $O(n-1)$  que sería igual a  $O(n)$

```

for (i= 0; i < n; i++) {
    int valido = 0;
    while (!valido) {
        printf("Actividad %d\n", i + 1);
        printf("    Inicio: ");
        scanf("%d", &actividades[i].inicio);
        printf("    Fin:    ");
        scanf("%d", &actividades[i].fin);

        if (actividades[i].inicio < 0 || actividades[i].fin < 0) {
            printf("    Error: los tiempos no pueden ser negativos.\n");
            printf("    Intente de nuevo.\n\n");
        } else if (actividades[i].inicio > actividades[i].fin) {
            printf("    Error: el inicio (%d) no puede ser mayor que el fin (%d).\n",
                actividades[i].inicio, actividades[i].fin);
            printf("    Intente de nuevo.\n\n");
        } else {
            valido = 1;
            printf("\n");
        }
    }
}

```

como se puede ver el bucle se realiza n veces

el while se realizará suponiendo que se ejecuta bien siempre

por lo que el while se considera  $O(1)$

entonces su complejidad sería  $O(n)$

Nota: las funciones u operaciones que no se muestran es porque pertenecen a la complejidad  $O(1)$

Concluyendo así que el de mayor complejidad computacional es el que prevalece por lo que nuestro algoritmo tiene complejidad  $O(n \log n)$

### 3.1.9. Aplicaciones

- Planificación de horario académico
- Selección de eventos de un calendario
- Organización de tareas para un proyecto

## 3.2. Mochila Fraccionaria:

Se tiene una mochila con capacidad máxima  $P$  y un conjunto de  $n$  objetos. Cada objeto " $i$ " tiene un valor " $v_i$ " y un peso " $p_i$ ". El objetivo es maximizar el valor total transportado sin exceder la capacidad máxima de la mochila

### 3.2.1. Estrategia voraz:

Nuestro criterio para solucionar este problema consiste en

1 Calcular la relación valor/peso de cada objeto

2 Ordenar los objetos de mayor a menor relación

3 Recorrer los objetos en ese orden:

    si el objeto cabe completo, tomarlo entero.

    si no cabe completo tomar la fracción que quepa y detenerse.

4 El algoritmo termina cuando la mochila se llena o se acaban los objetos

### 3.2.2. Elección local:

En cada paso, el algoritmo elige el objeto con mayor relación valor/peso entre los que aún no se han considerado, tomando así la mayor relación siempre.

### 3.2.3. Propiedad de elección voraz:

Si una solución óptima no tomara todo lo posible del objeto de mayor relación, se podría reemplazar parte de otro objeto de menor relación por una cantidad equivalente del de mayor relación, aumentando o manteniendo el valor total.

#### 3.2.4. Subestructura óptima

Siempre después de elegir cuánto tomar del objeto de mayor relación, queda un subproblema: llenar el resto de la capacidad con los objetos restantes, este subproblema tiene la misma estructura original: maximizar valor con una capacidad menor y menos objetos

#### 3.2.5. Corrección de la estrategia voraz:

Sea  $O$  una solución óptima

sea  $i$  el objeto de mayor relación valor/peso

Si  $O$  ya toma todo lo posible del objeto  $i$ , resuelto

Si  $O$  no toma todo lo posible de  $i$ , entonces toma alguna cantidad de otro objeto  $j$  con menor relación.

La diferencia de peso que falta del objeto  $i$  se puede compensar quitando esa misma cantidad de peso de los objetos de menor relación que  $O$  tiene de más y como  $i$  tiene mayor relación que los otros, el valor total no disminuye, repitiendo este intercambio, obtenemos una solución óptima que sí toma todo lo posible de  $i$ . por lo tanto, existe una solución óptima que incluye la elección greedy

#### 3.2.6. Diferencia con la mochila 0/1:

En la mochila 0/1 no se puede tomar fracciones de un objeto, se toma completo o no se toma. Esto impide hacer el intercambio peso por peso del argumento anterior porque no se puede ajustar exactamente la cantidad que falta, por eso el algoritmo greedy no garantiza solución óptima en el problema de la mochila 0/1

### 3.2.7. Implementación en C

```
=====
PROBLEMA DE LA MOCHILA FRACCIONARIA
=====
Ingrese la capacidad de la mochila: 100
Ingrese la cantidad de objetos: 10

Objeto 1 - Valor: 20
Objeto 1 - Peso: 5
```

Elegiremos la capacidad de la mochila, la cantidad de los objetos y, el valor y peso de cada objeto.

```
Objeto 1 - Valor: 20
Objeto 1 - Peso: 5

Objeto 2 - Valor: 30
Objeto 2 - Peso: 10

Objeto 3 - Valor: 15
Objeto 3 - Peso: 3

Objeto 4 - Valor: 50
Objeto 4 - Peso: 25

Objeto 5 - Valor: 40
Objeto 5 - Peso: 7
```

```
Objeto 6 - Valor: 10
Objeto 6 - Peso: 9

Objeto 7 - Valor: 25
Objeto 7 - Peso: 4

Objeto 8 - Valor: 80
Objeto 8 - Peso: 3

Objeto 9 - Valor: 76
Objeto 9 - Peso: 41

Objeto 10 - Valor: 24
Objeto 10 - Peso: 3
```

Finalmente nos muestra la ID que representa al objeto, su relación valor/peso, fracción de cuánto se tomó del peso

| MOCHILA FRACCIONARIA - RESULTADO |          |       |       |          |                 |
|----------------------------------|----------|-------|-------|----------|-----------------|
| ID                               | Relacion | Peso  | Valor | Fraccion | Peso en mochila |
| 8                                | 26.67    | 3.00  | 80.00 | 1.00     | 3.00            |
| 10                               | 8.00     | 3.00  | 24.00 | 1.00     | 3.00            |
| 7                                | 6.25     | 4.00  | 25.00 | 1.00     | 4.00            |
| 5                                | 5.71     | 7.00  | 40.00 | 1.00     | 7.00            |
| 3                                | 5.00     | 3.00  | 15.00 | 1.00     | 3.00            |
| 1                                | 4.00     | 5.00  | 20.00 | 1.00     | 5.00            |
| 2                                | 3.00     | 10.00 | 30.00 | 1.00     | 10.00           |
| 4                                | 2.00     | 25.00 | 50.00 | 1.00     | 25.00           |
| 9                                | 1.85     | 41.00 | 76.00 | 0.98     | 40.00           |
| VALOR TOTAL LOGRADO: 358.15      |          |       |       |          |                 |

### 3.2.8. Complejidad computacional

```
void imprimirResultado(Objeto objetos[], int n, double valorTotal) {
    printf("\n=====\\n");
    printf("    MOCHILA FRACCIONARIA - RESULTADO\\n");
    printf("=====\\n");
    printf(" ID | Relacion | Peso | Valor | Fraccion | Peso en mochila\\n");
    printf("-----+-----+-----+-----+-----+-----\\n");
    int i;
    for (i = 0; i < n; i++) {
        if (objetos[i].fraccion > 0) {
            printf(" %3d | %6.2f | %6.2f | %7.2f | %5.2f | %6.2f\\n",
                objetos[i].id,
                objetos[i].relacion,
                objetos[i].peso,
                objetos[i].valor,
                objetos[i].fraccion,
                objetos[i].peso * objetos[i].fraccion);
        }
    }

    printf("=====\\n");
    printf(" VALOR TOTAL LOGRADO: %.2f\\n", valorTotal);
    printf("=====\\n");
}
```

La función imprimir resultado tiene un valor de  $O(1)$  repetido  $n$  veces, es decir que su complejidad computacional es  $O(n)$

```

if (sumaPesos <= capacidad) {
    double valorTotal = 0.0;
    for (i = 0; i < n; i++) {
        objetos[i].fraccion = 1.0;
        valorTotal += objetos[i].valor;
    }
    printf("\nTodos los objetos caben en la mochila.\n");
    imprimirResultado(objetos, n, valorTotal);
    free(objetos);
    return 0;
}

```

Para el caso en que todos los objetos caben en la mochila igualmente su complejidad es  $O(n)$  al recorrer cada objeto uno por uno

```

qsort(objetos, n, sizeof(Objeto), compararObjetos);

```

Nosotros sabemos que la complejidad del ordenamiento qsort es de  $O(n \log n)$

```

for (i = 0; i < n; i++) {
    if (pesoActual + objetos[i].peso <= capacidad) {
        // Cabe completo
        objetos[i].fraccion = 1.0;
        pesoActual += objetos[i].peso;
        valorTotal += objetos[i].valor;
    } else {
        // Tomar fracción
        double espacioRestante = capacidad - pesoActual;
        objetos[i].fraccion = espacioRestante / objetos[i].peso;
        valorTotal += objetos[i].valor * objetos[i].fraccion;
        pesoActual = capacidad;
        break;
    }
}

```

Al tener los datos ordenados y recorrerlos uno por uno, el llenado de la mochila posee complejidad  $O(n)$  al solo recorrer dato por dato  $n$  veces

nuevamente al poder observar que la complejidad mayor es la del qsort la complejidad del algoritmo es de  $O(n \log n)$

#### 3.2.9. Aplicaciones:

- Mezcla de productos para medir las proporciones
- Seleccionar alimentos por valor peso/volumen
- Asignación de espacio de un lugar

### 3.3. Codificación de Huffman:

Algoritmo destinado a comprimir información presentada en forma de texto mediante un árbol de Huffman que construye un código binario variable donde se tiene un conjunto de símbolos y sus frecuencias asignando a cada símbolo una secuencia de bits de modo que los símbolos más frecuentes tengan códigos más cortos y los menos frecuentes códigos más largos.

#### 3.3.1. Estrategia voraz

- Paso 1: Para este algoritmo construiremos el árbol binario.
- Paso 2: Creamos un nodo hoja por cada símbolo, con su frecuencia
- Paso 3: Ordenamos los nodos en una cola de prioridad ordenada por frecuencia
- Paso 4: Creamos un nuevo nodo interno cuya frecuencia sea la suma de las dos anteriores

- Paso 5: Hacer que los dos nodos extraídos sean hijos izquierdo y derecho del nuevo nodo
- Paso 6: insertamos el nuevo nodo en la cola
- Paso 7: Repetimos este proceso hasta llegar a un último nodo
- Paso 8: El último nodo que queda es la raíz de árbol del cual partiremos en busca de cada uno de los símbolos fijándonos en las etiquetas de "0" y "1"

Observación: asignamos código izquierda =0, derecha=1 recorremos el árbol para obtener el código de cada símbolo

Ejemplo:

Tenemos la oración:

"La casa cercenada"

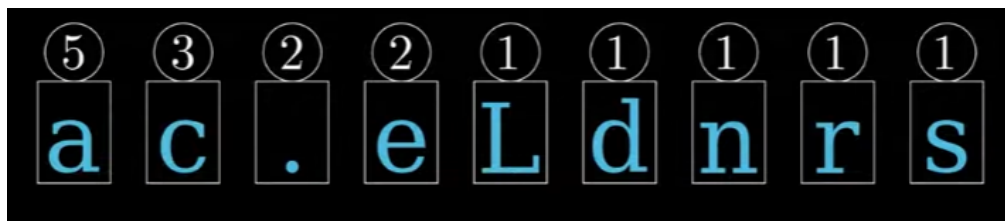
Al espacio le asignaremos el símbolo de "."

Paso 2:

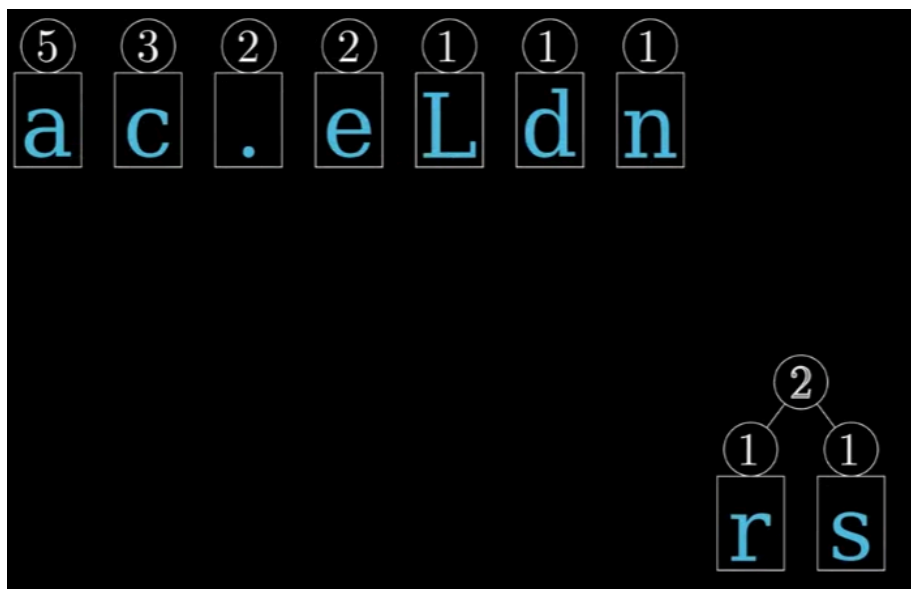
| Símbolo | Frecuencia |
|---------|------------|
|         |            |
| .       | 2          |
| L       | 1          |
| a       | 5          |
| c       | 3          |
| d       | 1          |
| e       | 2          |
| n       | 1          |

| Símbolo | Frecuencia |
|---------|------------|
|         |            |
| r       | 1          |
| s       | 1          |

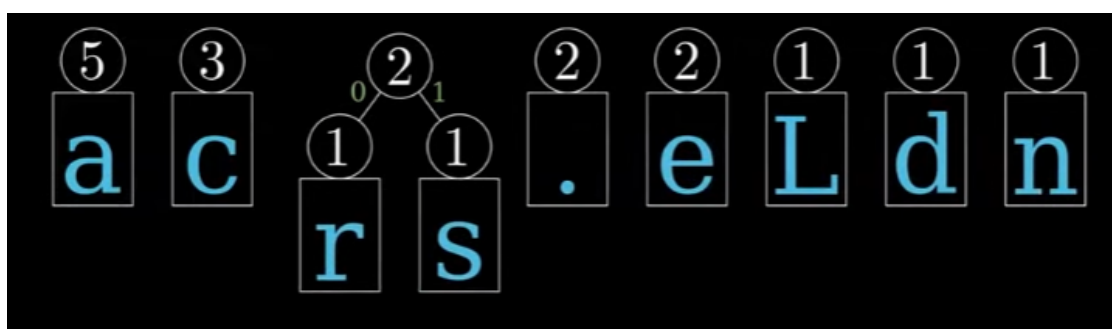
Paso 3: Ordenaremos la frecuencia de mayor a menor como se muestra en la imagen.



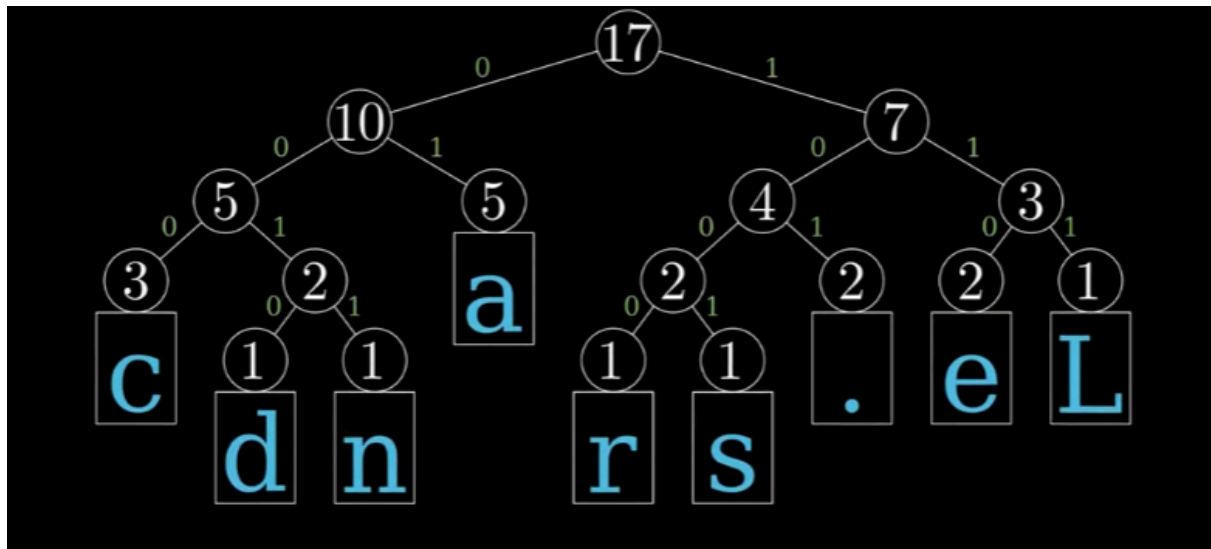
Realizaremos el Paso 4 y 5:



Paso 6:



Se realiza el Paso 7 hasta llegar al árbol de Huffman:



Paso 8:

c: 000  
d: 0010  
n: 0011  
a: 01  
r: 1000

s: 1001  
.: 101  
e: 110  
L: 111

### 3.3.2. Elección local:

En cada paso, el algoritmo elige los dos nodos de menor frecuencia para combinarlos

Esto es así porque los símbolos menos frecuentes deben quedar más profundo del árbol y los más frecuentes más cerca de la raíz. Al combinar los menos frecuentes primero aseguramos que estos queden al final

### 3.3.3. Propiedad de elección voraz

Existe una solución óptima en la que los dos símbolos de menor frecuencia tienen la misma profundidad máxima, al presentar la misma profundidad no resulta en cambios el intercalar uno o el otro

### 3.3.4. Subestructura óptima

Después de combinar los dos nodos de menor frecuencia, se crea un nuevo nodo con la suma de sus frecuencias. El problema restante es construir un árbol de Huffman para los

$n-1$

$n-1$  nodos restantes (incluyendo el nuevo nodo combinado).

Ese subproblema tiene exactamente la misma estructura que el original.

Si resolvemos óptimamente este subproblema y le agregamos la combinación inicial, obtenemos el árbol óptimo global

### 3.3.5. corrección de la estrategia voraz

Sea O un árbol de huffman óptimo

Sean x e y los dos símbolos de menor frecuencia

En o los nodos más profundos deben ser hojas, si x e y no son hermanos se pueden intercambiar con los hermanos de las hojas más profundas para que sí lo sean, sin aumentar el costo total, por lo tanto existe un árbol óptimo donde x e y son hermanos (hermanos significa si se han combinado para crear un nuevo nodo)

### 3.3.6. Implementación en C

```
=====
CODIFICACION DE HUFFMAN
=====
Ingrese una oracion o texto:
> Hola mundo

=====
CODIGOS DE HUFFMAN
=====
Simbolo | Frecuencia | Codigo | Longitud
-----+-----+-----+-----
(esp)   |          1 | 000    |        3
H       |          1 | 1110   |        4
a       |          1 | 011    |        3
d       |          1 | 1111   |        4
l       |          1 | 101    |        3
m       |          1 | 010    |        3
n       |          1 | 100    |        3
o       |          2 | 110    |        3
u       |          1 | 001    |        3
=====
Longitud total del mensaje codificado: 32 bits
Longitud original (8 bits por caracter): 80 bits
=====
```

Se insertará un texto y el programa mediante el árbol de Huffman modificará el texto para transformarlo en una secuencia de bits.

### 3.3.7. Complejidad

```
MinHeap* crearMinHeap(int capacity) {  
    MinHeap* heap = (MinHeap*)malloc(sizeof(MinHeap));  
    heap->size = 0;  
    heap->capacity = capacity;  
    heap->array = (Node**)malloc(capacity * sizeof(Node*));  
    return heap;  
}
```

La complejidad de esta función es  $O(n)$  porque está reservado un arreglo por cada cantidad de símbolos distintos del texto

### 3.3.8. Aplicaciones

- Compresión de archivos
- Compresión de imágenes
- Compresión de videos

## 4. Complejidad

Antes de ir directo al grano vamos a demostrar la complejidad del QuickSort y el Min-Heap binario ya que se usará posteriormente.

### ☐ QuickSort

Partimos de la recurrencia

$$T_{(n)} = 2T\left(\frac{n}{2}\right) + cn \quad \text{donde } cn \text{ representa el costo de particionar}$$

Expandimos

$$T_{(n)} = 2[2T(\frac{n}{4}) + c\frac{n}{2}] + cn$$

$$T_{(n)} = 4T(\frac{n}{4}) + 2cn$$

Una vez más

$$T_{(n)} = 4[2T(\frac{n}{8}) + c\frac{n}{4}] + 2cn$$

$$T_{(n)} = 8T(\frac{n}{8}) + 3cn$$

despues de k expansiones

$$T_{(n)} = 2^k T(\frac{n}{2^k}) + kcn$$

ahora hacemos el cambio  $n = 2^k$  y  $k = \log_2 n$

Sustituyendo

$$T_{(n)} = 2^{\log_2 n} T(1) + cn \log_2 n$$

Siendo  $T(1)$  constante d

$$T_{(n)} = nd + cn \log_2 n$$

$$T_{(n)} = O(n) + O(n \log n)$$

Aplicando límites

$$\lim_{x \rightarrow \infty} \frac{n \log n}{n} = \lim_{x \rightarrow \infty} \log n = \infty \quad \text{Concluimos que el término dominante es}$$

$n \log n$  ya que es el que crece más rápido

$$T_{(n)} = O(n \log n)$$

☐ Min-Heap binario

Primero hay que saber que un heap binario es un árbol binario completo

donde si su altura es  $h$  el número máximo de nodos es:

$$1 + 2 + 2^1 + 2^2 + 2^3 \dots + 2^h$$

La suma de una progresión geométrica

$$S = \frac{r^{h+1} - 1}{r - 1}$$

Como  $r=2$

$$S = \frac{2^{h+1} - 1}{2 - 1} = 2^{h+1} - 1$$

Ese S representaría a todos los nodos por ende le podemos dar n

$$n = 2^{h+1} - 1$$

$$n + 1 = 2^{h+1}$$

$$\log_2(n + 1) = \log_2 2^{h+1}$$

$$\log_2(n + 1) = h + 1$$

$$h = \log_2(n + 1) - 1$$

$$h = O(\log_2(n + 1)) \text{ El -1 es constante}$$

$$h = O(\log n)$$

$$T(n) = O(n) = O(\log n)$$

- Selección de actividades

**-Complejidad espacial:**

$$n = \text{número de actividades}$$

*Cómo usamos el quicksort para ordenar los datos*

$$T_{\text{quicksort}}(n) = O(n \log n) \text{ no lo vamos a resolver ya que lo explicó el profesor}$$

Empezamos con el recorrido Greedy (Ojo se ejecutó  $n - 1$  veces)

$S_i \geq f_{\text{última}}$  (en cada caso se hace una comparación)

$S_i = \text{tiempo de inicio de la actividad}$

$f_{\text{última}} = \text{tiempo de finalización de la última actividad}$

Por tanto

$$T_{\text{Greedy}}(n) = n - 1 \quad T_{\text{Greedy}}(n) = O(n)$$

Complejidad total

$$T(n) = T_{\text{Greedy}}(n) + T_{\text{quicksort}}(n)$$

$$T(n) = O(n) + O(n \log n)$$

Aplicando límites

$$\lim_{x \rightarrow \infty} \frac{n \log n}{n} = \lim_{x \rightarrow \infty} \log n = \infty \quad \text{Concluimos que el término dominante es}$$

$n \log n$  ya que es el que crece más rápido

Finalmente:  $T(n) = O(n \log n)$

### **-Complejidad espacial**

Para nuestro código

$\text{Actividad} = (\text{inicio}, \text{fin})$

para  $n$  actividades

$$M(n) = n \cdot \text{sizeof}(\text{Actividad})$$

Debido a que  $\text{sizeof}$  es una constante se puede quitar

$$M(n) = n \cdot c$$

Por tanto

$$M(n) = O(n)$$

Pero las variables ocupan memoria constantemente y sería  $O(1)$

$$M(n) = O(n) + O(1) \quad \text{obviamente el } O(n) \text{ es más dominante}$$

Finalmente

$$M(n) = O(n)$$

- Problema de mochila fraccionaria

### -Complejidad Temporal

$n$  = número de objetos

Calculando la relación valor/peso

$$r_i = \frac{v_i}{p_i}$$

Como se realiza una vez para cada objeto

$$T_1(n) = \sum_{i=1}^n 1 = n$$

$$T_1(n) = O(n)$$

Tenemos que comprobar si todos los objetos caben

$$T_2(n) = O(n)$$

Ahora el ordenamiento Greedy

$$\text{El criterio es } \frac{v_1}{p_1} \geq \frac{v_2}{p_2} \geq \dots \geq \frac{v_n}{p_n}$$

La complejidad del quicksort es

$$T_3(n) = O(n \log n)$$

Por último falta el llenado de mochila y en el peor de los casos recorre los  $n$  objetos.

$$T_4(n) = O(n)$$

Finalmente la complejidad total

$$T(n) = T_1(n) + T_2(n) + T_3(n) + T_4(n)$$

$$T(n) = O(n \log n) + O(3n)$$

Aplicando límites

$$\lim_{x \rightarrow \infty} \frac{n \log n}{3n} = \lim_{x \rightarrow \infty} \frac{\log n}{3} = \infty \text{ Concluimos que el término dominante es } n \log n$$

ya que es el que crece más rápido

$$T(n) = O(n \log n)$$

### -Complejidad Espacial

Cada objeto posee

$$\text{Objeto} = (id, valor, peso, relación, fracción)$$

Como todos tienen tamaño constante

$$M(n) = n \cdot \text{sizeof}(\text{objeto})$$

Debido a que sizeof es una constante se puede quitar

$$M(n) = n \cdot c$$

Por tanto

$$M(n) = O(n)$$

Pero claro las variables como sabemos tienen memoria

$$M(n) = O(n) + O(1)$$

$$M(n) = O(n)$$

- Código Huffman

### -Complejidad Temporal

$$n = \text{número de caracteres distintos de texto}$$

En cada iteración se toman 2 nodos y se crea 1

$$n \rightarrow n - 1$$

Como hemos empleado el Min-Heap en extraermin() su complejidad es

$$T_{extraerMin}(n) = O(\log n)$$

Cómo lo hacemos 2 veces

$$T_{extraerMin}(n) = 2O(\log n)$$

En la parte de inserción(subir) del código también se emplea Min-heap

$$T_{inserción}(n) = O(\log n)$$

En total sería

$$T(n) = T_{extraerMin}(n) + T_{inserción}(n)$$

$$T(n) = 2O(\log n) + O(\log n)$$

$$T(n) = 3O(\log n)$$

$$T(n) = O(\log n)$$

Pero como hay n-1 iteraciones

$$T(n) = (n - 1)O(\log n)$$

$$T(n) = O(n \log n - \log n)$$

Aquí se nota claramente que  $n \log n$  crece más rápido

$$T(n) = O(n \log n)$$

### **-Complejidad espacial**

Contamos los espacios

Min-heap:  $O(n)$

Árbol de Huffman: aproximadamente  $2n-1$  nodos  $O(n)$

Tabla de frecuencias: Almacena  $n$  caracteres  $O(n)$

Códigos de Huffman: Almacena un código por carácter  $O(n)$

Recursión para generar códigos: En el peor de los casos  $O(n)$

$$M(n) = 5O(n) = M(n) = O(n)$$

## 5. Comparación de Algoritmos:

| Criterio                                                                                                                                                                                                                                                                                                                                                                                                 | Selección de actividades                                                                                                                                            | Mochila fraccionaria                                                                                                     | Codificación de Huffman                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Objetivo                                                                                                                                                                                                                                                                                                                                                                                                 | Seleccionar el mayor número de actividades compatibles.                                                                                                             | Maximizar el número de objetos dentro de una capacidad limitada(se pueden usar fracciones).                              | Reducir el tamaño de los datos mediante códigos de longitud variable.                                                                                    |
| Elección Greedy                                                                                                                                                                                                                                                                                                                                                                                          | Debemos seleccionar la actividad que dure menos                                                                                                                     | Seleccionar el elemento con mejor relación(peso-objeto)                                                                  | Combinar los dos símbolos de menor frecuencia                                                                                                            |
| Propiedad de elección Greedy                                                                                                                                                                                                                                                                                                                                                                             | La actividad que termina antes deja mayor tiempo disponible para seleccionar otras actividades, por lo que esta elección puede formar parte de una solución óptima. | Elegir el mayor valor/peso permite obtener el máximo beneficio posible por cada unidad de capacidad disponible.          | Combinar los dos símbolos menos frecuentes puede formar parte de un árbol de codificación óptimo.                                                        |
| Subestructura óptima                                                                                                                                                                                                                                                                                                                                                                                     | Después de elegir una actividad, las actividades faltantes forman un subproblema que también puede resolverse óptimamente.                                          | Después de ocupar parte de la capacidad, el problema restante puede resolverse óptimamente con los elementos que quedan. | Después de combinar los dos nodos de menor frecuencia, se obtiene un problema reducido que conserva la posibilidad de construir una codificación óptima. |
| Complejidad temporal                                                                                                                                                                                                                                                                                                                                                                                     | $O(n \log(n))$                                                                                                                                                      | $O(n \log(n))$                                                                                                           | $O(n \log(n))$                                                                                                                                           |
| Complejidad espacial                                                                                                                                                                                                                                                                                                                                                                                     | $O(n)$                                                                                                                                                              | $O(n)$                                                                                                                   | $O(n)$                                                                                                                                                   |
| ¿Cuál tiene la mejor complejidad temporal y espacial?                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                     |                                                                                                                          |                                                                                                                                                          |
| En términos de complejidad, los tres algoritmos presentan un rendimiento y eficiencia similar, con una complejidad temporal de $O(n \log n)$ y espacial de $O(n)$ . Sin embargo, cada uno está diseñado para un problema diferente de acuerdo a nuestras necesidades, por lo que la elección del algoritmo depende principalmente de las características del problema y no únicamente de su complejidad. |                                                                                                                                                                     |                                                                                                                          |                                                                                                                                                          |

## 6. Conclusión

En conclusión, los algoritmos Greedy constituyen una estrategia eficiente para resolver determinados problemas(no todos) de optimización mediante la selección de la mejor alternativa disponible en cada etapa. A través del desarrollo de los algoritmos de selección de actividades, mochila fraccionaria y codificación de Huffman, se pudo comprobar cómo una decisión local adecuada puede conducir a una solución global óptima cuando el problema cumple con la propiedad de elección Greedy y la subestructura óptima.

Asimismo, se analizó que, aunque los tres algoritmos utilizan la misma estrategia, cada uno emplea un criterio de selección diferente según el problema que debe resolver. La selección de actividades prioriza la menor hora de finalización, la mochila fraccionaria prioriza la mayor relación valor/peso y Huffman combina los elementos de menor frecuencia. En términos de eficiencia, los tres presentan una complejidad temporal típica de  $O(n \log n)$  y una complejidad espacial de  $O(n)$ , aunque su aplicación y funcionamiento son diferentes.

Finalmente, se concluye que la estrategia Greedy permite obtener soluciones eficientes y relativamente sencillas de implementar, pero no puede aplicarse de manera general a cualquier problema de optimización. Es necesario comprobar que las decisiones locales realmente conduzcan a una solución óptima. Por ello, comprender sus propiedades, ventajas y limitaciones resulta fundamental para determinar cuándo esta estrategia es adecuada frente a otros enfoques algorítmicos.

